Integração de interface gráfica analítica de dados de planeamento AEC

Relatório Final do Projeto Integrador

João Pedro Monteiro de Castro



Licenciatura em Engenharia Informática e Computação

Tutor na U.Porto: Rogério Reis
Orientador na empresa/Proponente: João Ribeiro

27/06/2025

Conteúdo

1 Introdução **2**
- 1.1 Enquadramento . . . 2
- 1.2 Objetivos e resultados esperados . . . 2
- 1.3 Estrutura do relatório . . . 3

2 Metodologia utilizada e principais atividades desenvolvidas **3**
- 2.1 Metodologia utilizada . . . 3
- 2.2 Intervenientes, papéis e responsabilidades . . . 4
- 2.3 Atividades desenvolvidas . . . 4
  - 2.3.1 Fases de Trabalho . . . 5

3 Desenvolvimento da solução **6**
- 3.1 Requisitos . . . 6
  - 3.1.1 Requisitos Funcionais . . . 6
  - 3.1.2 Requisitos Não Funcionais . . . 7
  - 3.1.3 Restrições . . . 7
- 3.2 Arquitetura e tecnologias utilizadas . . . 8
- 3.3 Solução desenvolvida . . . 9
  - 3.3.1 Desenvolvimento em ambiente próprio . . . 10
  - 3.3.2 Análise da base de dados e API da BIMMS . . . 13
  - 3.3.3 Produto final integrado . . . 16
- 3.4 Validação . . . 17

4 Conclusões **17**
- 4.1 Resultados alcançados . . . 17
- 4.2 Lições aprendidas . . . 17
  - 4.2.1 Fases de Trabalho . . . 18
- 4.3 Trabalho futuro . . . 18
- 4.4 Impacto ODS . . . 19

1 Introdução

1.1 Enquadramento

Este relatório final documenta o trabalho realizado no âmbito do estágio curricular na empresa BIMMS, dedicado à integração de componentes gráficas analíticas de dados de planeamento no domínio AEC (Architecture, Engineering and Construction). A BIMMS é hoje um centro tecnológico de referência em Construção Digital, com atuação pioneira em Portugal na implementação da metodologia BIM e na investigação aplicada de novas tecnologias como IoT e Digital Twin, suportando processos de melhoria contínua em projetos de grande complexidade em toda a Europa. O problema central abordado neste estágio curricular encontra-se com a necessidade de disponibilizar aos utilizadores de uma plataforma web de gestão de projetos de AEC ferramentas de visualização e análise de indicadores, tais como tarefas, prazos, custos, entre outros, de forma intuitiva e interativa. A carência de interfaces que tornem acessíveis e exploráveis grandes volumes de dados operacionais compromete a eficiência na tomada de decisão e na monitorização de projetos de construção, motivando o desenvolvimento de componentes visuais modulares e integráveis numa arquitetura full-stack existente.

1.2 Objetivos e resultados esperados

O objetivo principal foi desenhar e implementar um conjunto de componentes interativos que exibem e permitem editar cronogramas de tarefas (gráfico de Gantt) diretamente numa plataforma web (ms4aeco.com). Os requisitos centrais foram:

- Apresentar, de forma clara, o estado e as datas de cada tarefas;
- Permitir a edição através de arrastar as barras no gráfico, com gravação imediata no servidor;
- Filtrar e agrupar tarefas por estado (em curso, concluída, atrasada).

No final do estágio estes componentes deveriam estar integrados num ambiente de teste, a funcionar com a base de dados real da empresa, garantindo desempenho e usabilidade superiores. Esperava-se ainda recolher feedback dos utilizadores internos para ajustar outros aspetos, tornando a interface o mais intuitiva possível.

1.3 Estrutura do relatório

O relatório organiza-se em quatro capítulos principais, de acordo com as diretrizes da UC Projeto Integrador:

- **Capítulo 1 – Introdução**: contextualização do trabalho, definição de objetivos e descrição da estrutura do relatório.
- **Capítulo 2 – Metodologia utilizada e principais atividades desenvolvidas**: descrição da metodologia adotada, dos intervenientes e das atividades executadas, acompanhado de cronograma e entregáveis.
- **Capítulo 3 – Desenvolvimento da solução**: especificação de requisitos, arquitetura e tecnologias escolhidas, apresentação da solução do ponto de vista do utilizador e resultados da validação.
- **Capítulo 4 – Conclusões**: resumo dos resultados alcançados, reflexões sobre lições aprendidas, propostas de trabalho futuro e eventual contributo para os Objetivos de Desenvolvimento Sustentável.

Cada capítulo foi estruturado de forma a permitir uma leitura lógica e coerente, assegurando que todos os critérios de completude, rigor técnico-científico e clareza requisitados pelas diretrizes sejam cumpridos.

2 Metodologia utilizada e principais atividades desenvolvidas

2.1 Metodologia utilizada

Para a condução deste estágio curricular adotou-se uma metodologia ágil de desenvolvimento iterativo, baseado em sprints quinzenais e reuniões semanais de acompanhamento com o Orientador na empresa/Proponente, João Ribeiro. Este modelo permitiu uma avaliação contínua do progresso, identificação precoce de obstáculos e adaptações flexíveis no âmbito do projeto. Como principais ferramentas de suporte à colaboração e controlo de versões, recorreu-se ao GitHub para gestão de repositório e Microsoft Teams para comunicação assíncrona entre os intervenientes.

2.2 Intervenientes, papéis e responsabilidades

O projeto envolveu os seguintes intervenientes:

- **João Pedro Monteiro de Castro (estagiário):** responsável pela análise de requisitos, conceção, implementação e validação dos componentes gráficos analíticos, bem como pela elaboração da documentação técnica.
- **Rogério Reis (tutor na U.Porto):** orientação académica, validação do alinhamento com os objetivos da UC e revisões periódicas do progresso.
- **João Ribeiro (orientador na BIMMS):** definição e priorização de requisitos funcionais, coordenação com a equipa de developers e consolidação de feedback.
- **Equipa de Desenvolvimento BIMMS:** integração e apoio full-stack dos componentes e suporte técnico.

2.3 Atividades desenvolvidas

As atividades foram organizadas ao longo do cronograma abaixo e geraram os seguintes entregáveis:

- **Familiarização com a plataforma AEC** (14/02/2025–21/02/2025): análise da arquitetura e levantamento de dados relevantes.
- **Elicitação de requisitos e definição de métricas** (28/02/2025–07/03/2025): reuniões com a equipa de desenvolvimento para identificar indicadores-chave, resultando num documento de definição de métricas.
- **Conceção de protótipos component-based** (07/03/2025–04/04/2025): desenvolvimento de provas de conceito, protótipos em Figma e validação inicial.
- **Implementação de interfaces gráficas interativas** (04/04/2025–02/05/2025): codificação e deploy de componentes em ambiente de staging.

- **Integração via API** (09/05/2025–16/05/2025): compatibilização dos serviços RESTful com o backend de produção.
- **Pipeline de testes de usabilidade** (16/05/2025–23/05/2025): definição e execução de testes com utilizadores, acompanhado de relatório de feedback.
- **Otimização e refatoração de código** (23/05/2025–30/05/2025): melhorias de performance e resolução de bugs críticos.
- **Documentação técnica** (06/06/2025–27/06/2025): elaboração de guias de utilização, mapeamento de procedimentos e documentação do fluxo de desenvolvimento.

Tabela 1: Cronograma de atividades desenvolvidas

Atividade	**Período**
Familiarização com a plataforma AEC	14/02/2025–21/02/2025
Elicitação de requisitos e definição de métricas	28/02/2025–07/03/2025
Conceção de protótipos component-based	07/03/2025–04/04/2025
Implementação de interfaces gráficas interativas	04/04/2025–02/05/2025
Integração via API	09/05/2025–16/05/2025
Pipeline de testes de usabilidade	16/05/2025–23/05/2025
Otimização e refatoração de código	23/05/2025–30/05/2025
Documentação técnica	06/06/2025–27/06/2025

2.3.1 Fases de Trabalho

Para facilitar a compreensão do fluxo de trabalho, o estágio pode ser resumido em três fases gerais:

1. **Investigação e recolha de dados:** levantamento de exemplos de diagramas Gantt em repositórios GitHub e ferramentas online, além da definição inicial de requisitos junto dos stakeholders. Durante a fase de investigação, foram consultados vários repositórios e exemplos online, dos quais se destacam:

- `GanttChartHyperLibrary` [1] – Biblioteca Gantt Chart para JavaScript e TypeScript com Angular, React e extensões Vue;
- `gantt-task-react` [2] – Componente de Gantt Chart para React;
- `gantt-schedule-timeline-calendar` [3] – Biblioteca JavaScript para diagramas de Gantt, cronogramas e linhas do tempo, com funcionalidades de agendamento e gestão de reservas, compatível com React, Angular, Vue e Svelte;
- `ts-gantt` [4] – Biblioteca simples para criar uma Gantt Chart combinada com uma task grid.

2. **Prototipação em ambiente local:** desenvolvimento de provas de conceito em Vue.js/D3.js e Chart.js num ambiente pessoal, testando interações (renderização SVG, drag & drop, filtros, ...) e avaliando desempenho e usabilidade. Nesta fase, os dados utilizados foram criados artificialmente para simular cenários de planeamento e teste.

3. **Implementação em ambiente da empresa/final:** integração dos protótipos na infraestrutura Docker/MySQL da BIMMS, utilizando a base de dados real da empresa, otimizações de performance e execução de testes de integração e usabilidade com a equipa interna.

3 Desenvolvimento da solução

3.1 Requisitos

Para orientar o desenho, implementação e validação da componente gráfica de Gantt integrada na plataforma web AEC, foram identificados os seguintes requisitos, com base no levantamento junto de stakeholders, na análise da documentação interna da BIMMS e na investigação/procura sobre o tema noutros repositórios.

3.1.1 Requisitos Funcionais

- **RF1 – Leitura de tarefas via API:** a aplicação deve obter a lista de tarefas de um projeto AEC através de chamadas RESTful ao endpoint `/tasks`, incluindo ID, título, datas de início/fim, estado e responsáveis.

- **RF2 – Geração de gráfico Gantt:** deve renderizar, no frontend, um diagrama de Gantt que exiba barras horizontais correspondentes à duração de cada tarefa, organizadas por ciclo de vida do projeto.
- **RF3 – Interação Drag&Drop:** o utilizador deve poder alterar datas de início e fim diretamente no gráfico, com atualização automática dos dados no backend.
- **RF4 – Filtragem e agrupamento:** a componente deve permitir filtrar tarefas por estado (a decorrer, concluída, atrasada) e agrupar por responsável ou fase do ciclo de vida.
- **RF5 – Marcação de marcos (milestones):** deve sinalizar entregas intermédias como marcos no diagrama, com ícones ou barras de espessura diferenciada.
- **RF6 – Visualização de dependências:** apresentar ligações (setas) entre tarefas que dependem umas das outras.
- **RF7 – Operações CRUD:** o componente deve suportar a criação (POST `/tasks`), leitura (GET `/tasks/{id}`), atualização (PUT `/tasks/{id}`) e eliminação (DELETE `/tasks/{id}`) de tarefas, assegurando consistência dos dados no backend.

3.1.2 Requisitos Não Funcionais

- **RNF1 – Desempenho razoável:** o gráfico deve carregar e permitir interação de tarefas sem atrasos perceptíveis.
- **RNF2 – Adaptabilidade:** a interface deve ajustar-se corretamente a ecrãs de dimensões variadas (desktop e tablet).
- **RNF3 – Simplicidade de instalação:** o componente deve poder ser integrado sem alterações significativas na configuração atual do projeto.

3.1.3 Restrições

- Só se podem usar as tecnologias já adotadas no projeto (TypeScript, Vue.js, D3.js).
- O backend deve continuar a servir dados via API REST existente, sem novos endpoints.

3.2 Arquitetura e tecnologias utilizadas

Para suportar a componente gráfica de Gantt e a sua integração na plataforma web AEC, definiu-se uma arquitetura em três camadas, conforme ilustrado abaixo:

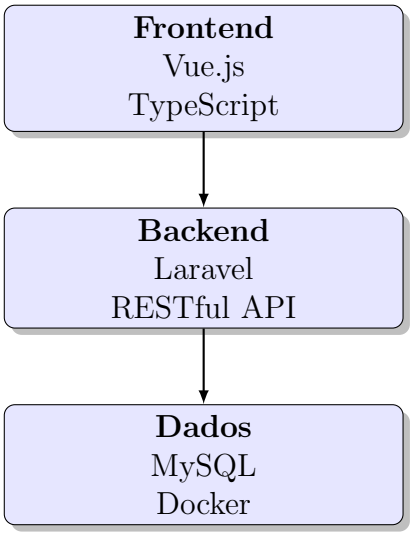


Figura 1: Diagrama de arquitetura em três camadas

- **TypeScript**: tipagem estática que reduz erros em tempo de desenvolvimento (vantagem quando comparado com o Javascript) e facilita reformações futuras.
- **Vue.js**: framework progressivo para construção de interfaces reativas, baseado em componentes, que permite isolar o gráfico de Gantt como módulo reutilizável.
- **D3.js**: fornece controlo avançado sobre elementos SVG para desenho de barras, setas e eventos de interação (drag & drop), essenciais para a visualização dinâmica das tarefas no desenvolvimento do projeto.
- **Chart.js**: utilizado como alternativa leve para a renderização de gráficos de suporte (por exemplo, histogramas de carga de trabalho) em contextos onde a simplicidade e rapidez de implementação são prioritárias.

- **Laravel**: framework PHP MVC que contem uma *RESTful API* robusta, com autenticação, políticas CORS e gestão de migrações, assegurando uma comunicação segura entre frontend e backend.
- **MySQL**: base de dados relacional escolhida pela sua integração nativa com Laravel, indexação eficiente de campos temporais e escalabilidade para grandes volumes de tarefas.
- **RESTful API**: definiu-se um conjunto de endpoints que seguem os princípios REST, garantindo interoperabilidade, cacheabilidade de respostas e facilidade de testes automatizados.

Dificuldades técnicas encontradas e resolução

Durante o desenvolvimento do componente surgiram dificuldades, que foram ultrapassadas através de investigação, documentação e recurso a ferramentas de inteligência artificial:

- **Vue.js e TypeScript:** sendo estas tecnologias novas para mim, foi necessário estudar a documentação oficial, explorar exemplos no GitHub e outros. Além disso, foi utilizado o ChatGPT para esclarecer dúvidas de sintaxe, compreender o ciclo de vida dos componentes e estruturar a comunicação entre themas.
- **Integração de eventos drag & drop:** a implementação de interações avançadas com D3.js revelou alguma complexidade na gestão de estados reativos e atualizações do DOM. Para resolver, foi também consultada informação online.

3.3 Solução desenvolvida

Nesta secção descreve-se todo o processo de criação, adaptação e integração da componente gráfico Gantt, seguindo três etapas principais: (i) desenvolvimento em ambiente local, (ii) análise e ajustamento à base de dados e API da BIMMS, e (iii) integração final e validação em ambiente de staging.

Repositório: O código-fonte está disponível em `https://github.com/joaopcastro710/Gantt`

3.3.1 Desenvolvimento em ambiente próprio

O projeto assenta numa arquitetura full-stack com Laravel (PHP) no backend e Vite + Vue.js (TypeScript) no frontend. Abaixo está um resumo das pastas e ficheiros mais relevantes:

`artisan` Interface de linha de comandos do Laravel, para executar migrations, testes e tarefas recorrentes.

`composer.json / composer.lock` Definição das dependências PHP e respetivas versões fixas.

`package.json / tsconfig.json / vite.config.js` Configuração do build e do ambiente de desenvolvimento do frontend (Vue + Vite).

`docker-compose.yml` Configuração dos serviços Docker (PHP, MySQL).

`README.md` Documentação de instalação, arranque e resolução de possíveis problemas.

`app/` Contém a lógica de negócio:

- `Http/Controllers`: controladores REST (e.g. `TaskActionController`, `LifecycleStageController`).
- `Models`: definições Eloquent (`TaskAction.php`, `TaskTemplate.php`, `LifecycleStage.php`, `User.php`).
- `Http/Middleware`: filtros de requisição (autenticação, CORS, etc.).

`config/` Ficheiros de configuração do Laravel (base de dados, mail, queue, etc.).

`database/`
- `migrations/`: definições de esquema (`users`, `task_templates`, `task_actions`, `lifecycle_stages`, ...).
- `seeders/` e `factories/`: geração de dados de teste.
- `gantt-seed.sql`: dumps SQL adicionais para cenários específicos.

`public/`
- `index.php`: front controller.
- `build/`: artefactos compilados pelo Vite (CSS, JS e `manifest.json`).

`resources/`
- `js/`: inicialização do Vue (`bootstrap.js`), componente principal (`App.vue`) e `components/GanttChart.vue`.
- `css/`: estilos globais (`app.css`).
- `views/`: templates Blade (layouts, `welcome.blade.php`).

`routes/`
- `web.php`: rotas para páginas Blade.
- `api.php`: endpoints JSON utilizados pelo componente Vue.

`tests/`
- `Feature/` e `Unit/`: respetivamente testes de integração e unitários.

`.env` : variáveis de ambiente — credenciais da BD, chaves API, URLs.

Pré-requisitos Para poder correr o projeto num ambiente local, são necessários:

- Docker e Docker Compose
- PHP $\geq$ 8.2
- Node.js e npm

Procedimento de instalação e arranque do projeto

1. **Via Docker Compose**
   (a) `Num terminal, e dentro da pasta do projeto, correr: docker-compose up -d`
2. **Backend Laravel**
   (a) Clonar o repositório.
   (b) Copiar `.env.example` para `.env` e configurar credenciais de acesso à Base de Dados.
   (c) `composer install` e `php artisan key:generate`.
   (d) `php artisan migrate --seed` para criar e povoar as tabelas.
   (e) Iniciar o servidor de desenvolvimento: `php artisan serve`.
3. **Frontend Vite/Vue**

(a) `npm install`

(b) `npm run dev`

Podemos concluir que na parte do Frontend, o ficheiro GanttChart.vue é o componente principal do Gantt. É o mais crucial para a representação visual do Gantt Chart que:

- Renderiza o gráfico usando D3.kjs para criar as barras visuais das tarefas;
- Implementa interatividade com drag & drop para mover e redimensionar tarefas;
- Gere filtros temporais (por dia, semana, mês);
- Controla modais para criar, editar e eliminar tarefas;
- Comunica com a API para operações CRUD;
- Organiza as barras para evitar sobreposição de tarefas do mesmo template.

Também temos o App.vue, a componente principal da aplicação Vue que serve como ponto de entrada e contém a estrutura base da aplicação.

Já na parte do Backend, podemos realçar:

- **TaskAction.php:** Representa as tarefas concretas no Gantt Chart com datas de início, fim e deadline, status da tarefas, relação com templates de tarefas e outros campos essenciais para o timeline Gantt;
- **TaskTemplate.php** Define os tipos de tarefas disponíveis, onde inclui o título e descrição das tarefas, a relação com as ações e serve como "molde"para criar novas tarefas no Gantt;
- **TaskActionController.php** Gere todas as operações CRUD das tarefas: listar tarefas com os templates associados, criar novas tarefas, atualizar datas e propriedades (drag & drop), eliminar tarefas e validação de dados;
- **TaskTemplateController** Fornece os templates disponíveis para criar novas tarefas no Gantt Chart.

O ficheiro api.php é um dos mais importantes também, já que é este que define os endpoints REST que o frontend consome.

Em seguida encontra-se uma screenshot de como ficou a versão final desta chart.

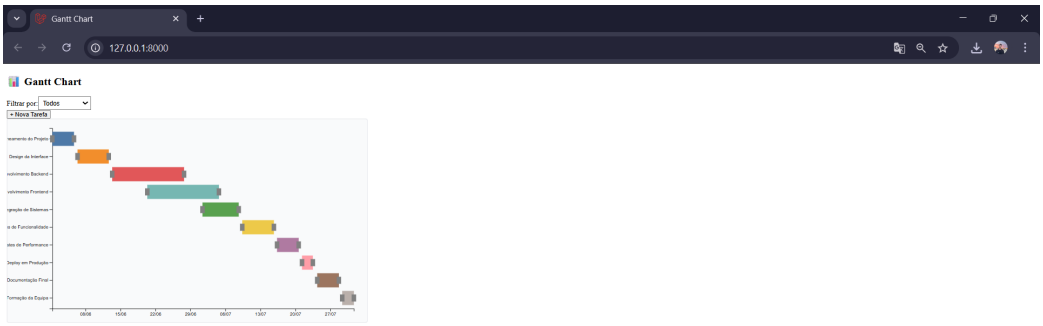


Figura 2: Versão inicial da Gantt Chart.

3.3.2 Análise da base de dados e API da BIMMS

Para garantir a compatibilidade do componente Gantt com o ambiente produtivo da BIMMS, realizou-se uma avaliação da base de dados e dos principais endpoints da API RESTful. Apresentam-se apenas os agrupamentos funcionais e os ajustes essenciais:

Visão modular da base de dados A base de dados da BIMMS encontra-se organizada em módulos funcionais, entre os quais se destacam:

- **Gestão de projetos:** tabelas de projetos, detalhes de projeto e associações de utilizadores.
- **Unidades e localizações:** subdivisão de projetos em unidades, com metadados adicionais armazenados em tabelas *details*.

- **Pessoas, organizações e agentes:** registos de utilizadores, perfis, estruturas organizacionais e definição de agentes com respetivos modelos de ação.
- **Controlo de acesso:** papéis, permissões e mapeamento para utilizadores e entidades de sistema.
- **Fases e ciclo de vida:** frameworks de projeto, fases, capítulos e detalhe de ciclo de vida associado a cada unidade.
- **Tarefas:** modelos de tarefas (`tasks_templates`), registo de ações de tarefa (`task_actions`) e ligações a unidades e agentes.
- **Entregáveis e artefactos:** definições de entregáveis, respetivas ações e gestão de ficheiros, miniaturas e ligações a elementos de projeto.

Durante a integração, foi necessário:

- Estender as consultas de `TaskAction` para incluir filtros por `project_id` e `unit_id`.
- Adaptar as associações a tabelas *details*, que contêm metadados críticos.
- Respeitar o modelo de separação entre "template" e "action" para não comprometer o histórico de registos.

Análise da API RESTful Foram identificados como mais relevantes os endpoints relacionados com `Task` e `LifeCycleDetails`, que suportam operações CRUD e audit trail necessárias ao componente de Gantt.

Tabela 2: Principais endpoints de `Task`

Método	Rota	Descrição
GET	/units/{unit_id}/tasks	Lista todas as *TaskActions* de uma unidade
GET	/tasks/{task_action_id}	Detalhes de uma ação de tarefa específica
POST	/tasks	Cria nova *TaskAction*.
POST	/tasks/{task_action_id}	Atualização (deveria ser `PATCH`, mas segue a convenção existente)
DELETE	/tasks/{task_action_id}	Remove uma *TaskAction*
GET	/tasks/{task}/elements	Lista elementos vinculados a um modelo de tarefa

Tabela 3: Principais endpoints de `LifeCycleDetails`

Método	Rota	Descrição
GET	/units/{unit_id}/lifecycle	Lista detalhes de ciclo de vida por estágio
PATCH	/units/{unit_id}/lifecycle	Atualização em lote de datas e orçamentos
GET	/units/{unit_id}/lifecycle/logs	Histórico de alterações (audit trail)
GET	/units/{unit_id}/lifecycle/{id}/notes	Anotações associadas a um detalhe específico
PATCH	/units/{unit_id}/lifecycle/{id}	Atualização de um único detalhe de ciclo de vida

Ajustamentos realizados Para contornar a ausência de endpoints `PUT`/`PATCH` padrão em `Task`, implementou-se uma rotina interna que executa `POST` em conformidade com a API existente, garantindo coerência transacional e cumprimento das regras de controlo de acesso definidas.

3.3.3 Produto final integrado

Após a fase de desenvolvimento local e a parametrização da API/BD da BIMMS, o componente de Gantt foi implantado no ambiente de staging da empresa. As principais funcionalidades disponíveis são:

- **Carregamento dinâmico de dados reais:** as tarefas são obtidas em tempo real através do endpoint `GET /units/{unit_id}/tasks`, respeitando filtros de projeto, unidade e fase.
- **Edição inline com persistência imediata:** alterações de datas de início/fim via drag & drop disparam um `POST /tasks/{task_action_id}`, garantindo atualização consistente no backend.
- **Integração responsiva:** o componente adapta-se a diferentes larguras de ecrã (desktop e tablet), mantendo usabilidade e clareza visual.



Figura 3: Visão do produto final: Gantt Chart totalmente integrado no ambiente da BIMMS.

Com este resultado, a plataforma passa a dispor de uma ferramenta robusta de planeamento visual, suportando decisões informadas e acompanhamento contínuo dos projetos de construção.

3.4 Validação

A validação da solução desenvolvida foi conduzida de forma iterativa, através de reuniões semanais de demonstração junto do orientador na BIMMS, onde se apresentou o protótipo em funcionamento e se recolheu feedback sobre usabilidade, performance e cobertura dos requisitos definidos.

4 Conclusões

4.1 Resultados alcançados

O projeto atingiu com sucesso os objetivos inicialmente definidos. Foi implementada e integrada uma componente gráfica Gantt, capaz de ler, apresentar e editar tarefas via API RESTful. Estão disponíveis funcionalidades de filtragem (dia, semana e mês), tudo testado em ambiente de staging com a base de dados criada, e em seguida com uma adaptação para a da empresa.

4.2 Lições aprendidas

Este estágio representou a minha primeira experiência profissional em desenvolvimento de software, proporcionando-me um crescimento acelerado em diversas vertentes. A adoção de uma metodologia ágil, com sprints quinzenais e revisões semanais, ensinou-me a importância de planear entregas de valor incrementais e de manter uma comunicação constante com o orientador, de modo a reagir rapidamente a novos requisitos ou descobertas. Inserido num ambiente de trabalho real na BIMMS, apercebi-me de como as boas práticas de revisão de código, controlo de versões e integração contínua elevam a qualidade final de um produto e reforçam a confiança na equipa. Do ponto de vista técnico, mergulhar em tecnologias até então desconhecidas — Vue.js, TypeScript,D3.js, entre outras — foi desafiante: aprendi a consultar documentação oficial, a explorar exemplos em repositórios GitHub e a usar melhor ferramentas de inteligência artificial como o ChatGPT para esclarecer dúvidas complexas de sintaxe e arquitetura. Por fim, percebi o valor de manter um canal de feedback aberto com todos os intervenientes, assegurando que cada decisão técnica se alinha com os objetivos do projeto e evita retrabalho desnecessário. Esta combinação de competências de planeamento, colaboração e investigação técnica constitui, sem dúvida, a principal lição que levarei para a minha carreira.

4.2.1 Fases de Trabalho

Para facilitar a compreensão do fluxo de trabalho, o estágio pode ser resumido em três fases gerais:

1. **Investigação e recolha de dados:** levantamento de exemplos de diagramas Gantt em repositórios GitHub e ferramentas online, além da definição inicial de requisitos junto dos stakeholders.

2. **Prototipação em ambiente local:** desenvolvimento de provas de conceito em Vue.js/D3.js e Chart.js num ambiente pessoal, testando interações (renderização SVG, drag & drop, filtros, ...) e avaliando desempenho e usabilidade. Nesta fase, os dados utilizados foram criados artificialmente para simular cenários de planeamento e teste.

3. **Implementação em ambiente da empresa/final:** integração dos protótipos na infraestrutura Docker/MySQL da BIMMS, utilizando a base de dados real da empresa, otimizações de performance e execução de testes de integração e usabilidade com a equipa interna.

4.3 Trabalho futuro

Nesta secção, apresentam-se duas linhas de evolução recomendadas para continuidade do projeto:

1. **Implementação do design visual desejado:** desenvolver a interface conforme o ficheiro de design, assegurando a fidelidade estética, consistência de estilos e melhoria da experiência de utilizador.

2. **Desenvolvimento de módulo de visualização de prazos (Deadline Chart):** avançar na criação de um componente complementar inspirado no Deadline Chart, com destaque para:

    - Visualização clara do estado antes e após o prazo;
    - Integração de indicadores de estado geral e milestones.

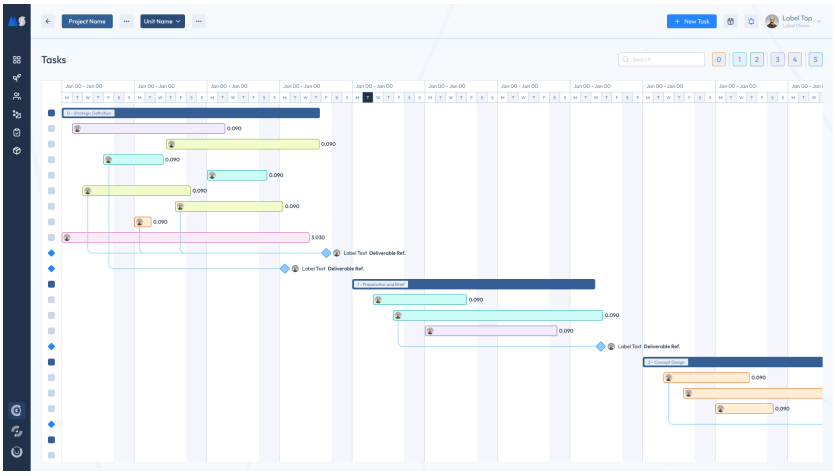


Figura 4: Design visual desejado para a interface Gantt.



Figura 5: Módulo de visualização de prazos (Deadline Chart) a desenvolver.

4.4 Impacto ODS

O presente trabalho enquadra-se diretamente nos Objetivos de Desenvolvimento Sustentável (ODS) das Nações Unidas, contribuindo de forma tangível para quatro metas prioritárias:

- **ODS 9 – Indústria, Inovação e Infraestrutura**: A integração de componentes gráficas analíticas no planeamento AEC fomenta a modernização digital de processos históricos, elevando o grau de automação e análise em tempo real. Isto promove infraestruturas mais resilientes, com capacidade de adaptação dinâmica a alterações de cronograma ou recursos, e incentiva a investigação contínua em tecnologias emergentes.

- **ODS 11 – Cidades e Comunidades Sustentáveis**: A melhoria na monitorização e gestão de projetos de construção contribui para a criação de espaços urbanos otimizados. O uso de indicadores visuais e alertas de prazos reduz o risco de atrasos ou falhas, minimizando impactos ambientais e sociais associados a trabalhos incompletos ou mal planeados, e reforçando práticas construtivas mais responsáveis.
- **ODS 12 – Consumo e Produção Responsáveis**: Ao antecipar necessidades de materiais e mão-de-obra, a interface permite uma alocação mais eficiente de recursos, reduzindo desperdícios, retrabalhos e custos associados. A visibilidade clara dos marcos e dependências possibilita planeamentos sustentáveis, com impactos reduzidos no uso de energia e matérias-primas.
- **ODS 17 – Parcerias para os Objetivos**: Este projeto é resultado de uma cooperação entre academia (Universidade do Porto), indústria (BIMMS) e comunidade open-source, exemplificando o poder das parcerias para acelerar a implementação de soluções inovadoras. A partilha de código, metodologias e feedback fortalece redes de colaboração que podem ser replicadas noutros contextos.

Adicionalmente, a modularidade e reusabilidade dos componentes promovem a disseminação destas ferramentas em projetos de menor escala ou em países em desenvolvimento, ampliando o impacto positivo além do contexto inicial. A continuidade deste trabalho poderá ainda incluir a internacionalização da interface, facilitando a adoção em línguas diferentes e em mercados diversos, ampliando o alcance destas boas práticas de planeamento sustentável.

Referências

[1] DlhSoftTeam. Ganttcharthyperlibrary: Biblioteca de gantt chart para javascript e typescript (angular, react, vue). `https://github.com/DlhSoftTeam/GanttChartHyperLibrary`, 2024. Acedido em: 27 de junho de 2025.

[2] MaTeMaTuK. gantt-task-react: Componente de gantt chart para react. `https://github.com/MaTeMaTuK/gantt-task-react`, 2024. Acedido em: 27 de junho de 2025.

[3] neuronetio. gantt-schedule-timeline-calendar: Biblioteca para diagramas de gantt, cronogramas e linhas do tempo. `https://github.com/neuronetio/gantt-schedule-timeline-calendar`, 2024. Acedido em: 27 de junho de 2025.

[4] Dmitry (yermolim) Yermolov. ts-gantt: Biblioteca para diagramas de gantt combinados com grid de tarefas. `https://github.com/yermolim/ts-gantt`, 2024. Acedido em: 27 de junho de 2025.